



**COMSATS University Islamabad,
Sahiwal Campus.**

EzHealth

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2021-2025)

By

**Muhammad Talha
Muhammad Mahad**

**CIIT/FA21-BCS-114/SWL
CIIT/FA21-BCS-120/SWL**



**COMSATS University Islamabad,
Sahiwal Campus.**

EzHealth

By

**Muhammad Talha
Muhammad Mahad**

**CIIT/FA21-BCS-114/SWL
CIIT/FA21-BCS-120/SWL**

Supervisor

Mr. Muhammad Jamil

Bachelor of Science in Computer Science (2021-2025)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of other

Declaration

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Talha

Muhammad Mahad

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**EzHealth**” was developed by **Muhammad Talha**(CIIT/FA21-BCS-114/SWL) and **Muhammad Mahad** (CIIT/FA21-BCS-120/SWL) under the supervision of “**Me. Muhammad Jamil**” and co-supervisor “**Mr. Arslan Sarwar**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. Muhammad Jamil

Lecturer

Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

EzHealth is a groundbreaking healthcare platform crafted to transform medical diagnostics by ensuring vital health services are accessible to everyone, regardless of their location or physical limitations. This innovative system seeks to redefine healthcare delivery by providing user-friendly diagnostic tools that empower patients to manage their health effortlessly. Offering the flexibility to monitor well-being anytime, anywhere, EzHealth caters to a wide range of schedules and needs, making healthcare more inclusive.

Developed with Kotlin for the mobile app, Laravel for the admin panel, and Firebase for real-time data management, EzHealth integrates Tailwind CSS to deliver a responsive and intuitive interface. Enhanced with robust features like secure user authentication, appointment scheduling, and report access, the platform ensures high performance and scalability. Administrators benefit from comprehensive tools to oversee users, appointments, and tests, streamlining healthcare operations with efficiency and precision.

The core mission of EzHealth is to eliminate barriers to diagnostics, leveraging modern technologies to provide a seamless, affordable, and high-quality health experience. By empowering users to take charge of their health and access critical services remotely, EzHealth is poised to reshape the healthcare landscape, fostering a generation of informed individuals who can prioritize their well-being and unlock new opportunities for a healthier life.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Muhammad Jamil” and our Co- Supervisor “Mr. Arslan Sarwar”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Talha

Muhammad Mahad

Abbreviations

Abbreviation	Meaning
LLM	Large Language Model
UC	Use Case
SDLC	Software Development Life Cycle
API	Application Programming Interface
CSS	Cascading Style Sheets
St	Streamlit

Table of Contents

1	Introduction.....	1
1.1	Brief.....	1
1.2	Relevance to Course Modules.....	1
1.2.1	Web Technologies (CSC312).....	1
1.2.2	Human-Computer Interaction (CSC356)	1
1.2.3	Operating Systems (CSC322)	2
1.2.4	Database Systems I (CSC371)	2
1.3	Project Background	2
1.4	Literature Review.....	2
1.4.1	Analysis from Literature Review.....	3
1.5	Methodology and Software Lifecycle for This Project.....	4
1.5.1	Rationale behind Selected Methodology	5
2	Problem Definition	8
2.1.1	Problem Statement	8
2.2	Deliverables and Development Requirements.....	9
2.3	Current System	11
3	Requirement Analysis.....	13
3.1	Use Case Description.....	14
3.2	Functional Requirements	23
3.3	Non-Functional Requirements.....	24
4	Design and Architecture.....	26
4.1	System Architecture.....	26
4.2	Data Representation [Diagram + Description]	27
4.3	Process Flow/Representation.....	28
4.4	Sequence Diagram.....	31
5	Implementation	32
5.1	Algorithm	34
5.2	Pseudo Code	35
5.3	External APIs	35
5.4	User Interface	36
6	Testing and Evaluation	47
6.1	Manual Testing.....	47
6.2	System testing.....	47

6.3	Unit Testing	47
6.4	Functional Testing	49
6.5	Automated Testing:.....	51
7	Conclusion and Future Work	52
7.1	Conclusion.....	52
7.2	Future Work.....	52

List of Figures

Figure 3.1. Use Case Diagram	13
Figure 4.1: Architecture Diagram	28
Figure 4.2: Flow Diagram.....	30
Figure 4.3: Sequence Diagram.....	31
Figure 4.4: Class Diagram	32
Figure 5.1: UML Diagram	34
Figure 5.2:DashBoard Screen	37
Figure 5.3 :Appointments Screen	38
Figure 5.4: BMI Screen.....	39
Figure 5.5: ChatBot.....	41

List of Tables

Table 1. Login.....	4
Table 2 Verify Account	4
Table 3 View Reports and Summary.....	5
Table 4 View Receipts.....	6
Table 5 View Test.....	6
Table 6 Add Appointment	6
Table 7 Check Status	7
Table 8 Web Reports	7
Table 9 Calculate BMI	8
Table 10 Emergency call	8
Table 11 Manage Reports.....	9
Table 12 Manage Users	9
Table 13 Manage Admins.....	10
Table 14 Manage Appointments.....	10
Table 15 Manage Tests	10
Table 16 Manage WebID and Pass.....	10
Table 17 Manage Logout.....	10
Table 18 Details of APIs Used	35
Table 19 Unit Testing 1	47
Table 20 Unit Testing 2	48
Table 21 Unit Testing 3	49
Table 22 Functional Testing 1	49
Table 23 Functional Testing 2	50
Table 24 Integration Testing	51
Table 25 Automated Testing	52

Chapter 1

Introduction

1 Introduction

The EzHealth platform is a comprehensive digital healthcare solution aimed at simplifying the access and management of medical diagnostic services. With the ongoing growth of mobile technology and web applications, the need for a centralized, accessible, and user-friendly health management system has become essential. EzHealth seeks to address these needs by providing users with the ability to view their health reports, schedule appointments, and access health metrics, while empowering administrators with robust tools for oversight and management of users, appointments, and reports.

1.1 Brief

The EzHealth platform is a comprehensive digital healthcare solution aimed at simplifying the access and management of medical diagnostic services. With the ongoing growth of mobile technology and web applications, the need for a centralized, accessible, and user-friendly health management system has become essential. EzHealth seeks to address these needs by providing users with the ability to view their health reports, schedule appointments, and access health metrics, while empowering administrators with robust tools for oversight and management of users, appointments, and reports.

1.2 Relevance to Course Modules

This project directly aligns with several key course modules:

1.2.1 Web Technologies (CSC312)

From studying web technologies, we learned the diverse aspects of website development, ranging from basic static pages to complex web applications. This understanding was vital in crafting my project's online platform, encompassing various web-based functionalities.

1.2.2 Human-Computer Interaction (CSC356)

Aligning with this course, my project prioritizes user-friendly interfaces across system modules. By integrating principles of human-computer interaction, I aim to create an intuitive system. This project application allows me to implement and assess user interface design, refining my skills in usability and user experience testing.

1.2.3 Operating Systems (CSC322)

Understanding operating systems was crucial for my project, as it forms the backbone of computer applications. We comprehended how applications interact with an operating system through an application program interface (API), contributing to a better grasp of system-level interactions.

1.2.4 Database Systems I (CSC371)

My project serves as a practical application of database management concepts learned in this course. It enables hands-on experience in designing and implementing databases, allowing me to translate theoretical database knowledge into real-world applications, emphasizing data storage and manipulation skills.

1.3 Project Background

In today's fast-paced world, many individuals struggle to maintain regular health checkups due to time constraints, limited access to medical facilities, or lack of awareness. This delay in diagnosis often leads to the progression of preventable diseases. Blood test reports, though widely available, are often underutilized by the general public due to the complexity of medical interpretation.

EzHealth addresses this gap by introducing an AI-powered mobile application that leverages machine learning to predict potential health issues based on blood test parameters. By integrating a pre-trained machine learning model into a user-friendly Android app, EzHealth empowers users to monitor their health conveniently and independently. This system not only promotes early detection but also supports preventive healthcare, especially in remote or underserved regions where access to medical expertise is limited.

The project blends mobile technology, artificial intelligence, and healthcare data to create an accessible diagnostic assistant, aimed at improving public health awareness and proactive disease management.

1.4 Literature Review

In recent years, the integration of Artificial Intelligence (AI) into healthcare

systems has gained significant momentum, especially in diagnostic applications. Machine learning (ML) models have been widely used for disease prediction, medical imaging analysis, and patient monitoring. Studies have shown that AI-based diagnostic tools can achieve comparable accuracy to professional medical assessments when trained on quality datasets.

A growing number of mobile applications and research prototypes have emerged that utilize health data for prediction and analysis. For example, IBM Watson Health has demonstrated the potential of AI in medical diagnostics, while other mobile apps like Ada and Babylon Health provide symptom checking based on user input. However, these tools often depend on internet connectivity, complex symptom queries, or subscription services, limiting their usability in offline and resource-constrained environments.

Several research papers have explored disease prediction using blood test parameters. Commonly predicted conditions include diabetes, anemia, liver disorders, and cardiovascular diseases. These studies typically employ supervised learning techniques such as Random Forest, Support Vector Machines (SVM), and Neural Networks, achieving accuracy scores above 85% when trained on labeled clinical datasets.

Despite these advancements, there is still a gap in providing **offline, low-cost, and user-friendly solutions** to the general population. Most existing applications lack the ability to run ML models natively on mobile devices or require cloud computation, which may pose privacy and accessibility concerns.

EzHealth bridges this gap by embedding a TensorFlow Lite model directly into an Android application, allowing offline predictions from blood test results without compromising data privacy or requiring constant internet access. This positions the project uniquely within the landscape of AI-based healthcare solutions.

1.4.1 Analysis from Literature Review

The literature review reveals that while AI and machine learning are actively being used in healthcare, especially for disease prediction and diagnostics, most

implementations are either web-based or require cloud computation. These solutions, though powerful, often depend on stable internet connectivity and may raise concerns about data privacy and real-time access.

In contrast, **EzHealth** stands out by offering a lightweight, mobile-based diagnostic assistant that functions **completely offline**. Unlike existing tools like Ada or Babylon Health which rely on symptom input and internet-based inference, EzHealth uses a **trained TensorFlow Lite model** that interprets actual **blood test parameters**—a more objective and reliable data source.

Furthermore, while many studies use machine learning for specific disease prediction (e.g., diabetes or heart disease), EzHealth adopts a **multi-class classification approach**, enabling it to predict multiple potential conditions based on comprehensive blood profiles. This provides a more holistic health overview rather than focusing on a single diagnosis.

The analysis highlights a key innovation: **bringing clinical-level AI diagnostics to the user's pocket, even in low-resource settings**. This democratizes access to healthcare insights and fills the gap between high-tech healthcare tools and everyday user accessibility. It also emphasizes privacy by ensuring that sensitive health data never leaves the device.

Thus, EzHealth not only aligns with current research trends but extends them into a **practical, accessible, and privacy-focused solution**, offering a meaningful contribution to AI-driven preventive healthcare.

1.5 Methodology and Software Lifecycle for This Project

The development of EzHealth followed a structured and iterative approach, ensuring continuous refinement of features based on testing and feedback. The selected Software Development Life Cycle (SDLC) model for this project is the **Incremental Model**.

In the **Incremental SDLC Model**, the system is designed, implemented, and tested in incremental builds, allowing core functionalities to be developed first and enhanced over successive versions. This approach was ideal for EzHealth, as it enabled early deployment of basic features like UI setup and blood parameter

input, followed by the integration of machine learning inference, offline support, and Firebase-based update notification.

Key Phases of the Methodology

1. Requirement Analysis

Requirements were gathered from the proposal, academic guidelines, and user expectations. The focus was on simplicity, offline capability, and accurate disease prediction.

2. SystemDesign

A modular architecture was designed. The frontend was built using **Kotlin (XML-based UI)**, while the backend model was developed and converted from **TensorFlow to TensorFlow Lite** for Android integration.

3. Implementation

The application was implemented in multiple builds. The first version handled UI and navigation, followed by data input validation, then ML model integration and finally Firebase update checking.

4. Testing

Each increment was tested individually using real-world sample data. Emphasis was placed on correctness of ML predictions, usability, and offline functionality.

5. Deployment & Maintenance

The app was packaged as an APK and made available through an update link. Maintenance includes retraining the model for future diseases and adding new blood test parameters.

This methodology allowed flexibility and ensured each component met quality and performance standards before integration. It also enabled parallel development of the Android app and the ML model.

1.5.1 Rationale behind Selected Methodology

The selection of both the **Incremental SDLC model** and the **Object-Oriented**

Programming (OOP) methodology was based on the specific needs and nature of the EzHealth project.

Why Incremental SDLC?

EzHealth was developed in multiple functional parts—user interface, data entry, machine learning model integration, offline processing, and Firebase updates. These parts required isolated development and testing before being merged. The **Incremental Model** supported this well by allowing:

- **Early delivery of core functionality**, such as a basic UI and input form.
- **Progressive integration** of advanced features like ML prediction and Firebase sync.
- **Continuous feedback and refinement**, ensuring that each build met functional requirements.
- **Reduced risk**, as critical components were tested early before full integration.

This model also made it easier to manage complexity and track progress in a project with clear modular components.

Why Object-Oriented Methodology?

The app was developed in **Kotlin**, which inherently supports **Object-Oriented Programming**. This approach was chosen due to its benefits in structuring and maintaining the application:

- **Modularity**: Different components such as input forms, prediction handlers, and database utilities were developed as separate classes for better organization.
- **Reusability**: Code for reading input, validating parameters, and displaying results could be reused across different activities.
- **Scalability**: Future enhancements (e.g., adding new prediction types or reports) can be integrated with minimal code changes.
- **Maintainability**: Debugging and updates are easier with well-structured, class-based architecture.

By combining the **Incremental SDLC model** with **OOP principles**, EzHealth was developed in a clean, manageable, and extensible manner—ideal for both academic and real-world applications.

Chapter 2

Problem Definition

2 Problem Definition

EzHealth is a mobile application designed to provide personalized medical recommendations based on blood test data. The application leverages advanced machine learning algorithms to predict potential health issues and offer preventive suggestions by analyzing key biomarkers from blood tests.

Currently, patients often receive their blood test results without comprehensive understanding or actionable insights, leading to delayed diagnosis or inadequate healthcare management. Medical professionals may not always have immediate access to the context of a patient's complete medical history, making it harder to provide timely advice.

EzHealth aims to address these challenges by enabling users to easily input their blood test data, which will then be processed through a trained machine learning model, offering actionable medical insights and health recommendations directly from their mobile device.

Outcome

EzHealth addresses this problem by delivering a mobile application that uses an embedded **TensorFlow Lite machine learning model** to predict potential diseases based on blood test input. The app is designed to function **completely offline**, ensuring accessibility even in remote areas, and provides users with **instant insights** about their health in a clear, non-technical format.

This solution bridges the gap between clinical diagnosis and user autonomy, promoting **preventive healthcare**, improving **health literacy**, and enabling **early detection** of health issues—without depending on professional consultation or internet services.

2.1.1 Problem Statement

Access to timely and accurate medical diagnosis remains a major challenge, particularly in under-resourced or remote areas where healthcare services are limited. Blood tests are among the most common diagnostic tools, yet interpreting the results typically requires a professional consultation, which may

not be immediately accessible or affordable for many individuals.

Despite the increasing availability of digital healthcare solutions, most require internet connectivity or rely on cloud-based machine learning inference, raising concerns about **data privacy**, **latency**, and **accessibility**. Existing mobile applications often fail to provide **offline**, **instant**, and **user-friendly** diagnostic support based on objective medical data like blood test parameters.

The problem to be solved, therefore, is

"How can we empower users to interpret their blood test results and receive early health risk assessments—without internet access or medical expertise—while maintaining user privacy and accuracy?"

2.2 Deliverables and Development Requirements

Deliverables and development requirements.

Deliverables

1. Mobile Application (Android)

- A fully functional **Android app** built using **Kotlin**.
- Integration of the **TensorFlow Lite model** for processing blood test data and providing personalized medical recommendations.
- A **user-friendly interface** that allows users to input and view their blood test data and health recommendations.
- **Offline functionality** to store blood test data and recommendations locally, with the ability to sync when online.
- Push **notifications** for health reminders and alerts (e.g., abnormal test results).
- **Graphical representations** of test results (e.g., bar charts, line graphs).
- **User Authentication** using Firebase or Laravel for user login and profile management.

2. Laravel Admin Panel

- Manage user data, available tests and medical reports.
- Add, update, or delete medical reports
- Handle appointment scheduling and maintain records of user health consultations.

- Oversee system activity and manage backend services such as report metadata

3. Machine Learning Model

- A **TensorFlow Lite model** that processes blood test data (e.g., glucose, cholesterol) and outputs health recommendations.
- Periodic updates to the model, with retraining based on more data or improvements in prediction accuracy.

4. Documentation

- **User Manual** with instructions on how to use the app, enter data, and interpret health recommendations.
- **Technical Documentation** detailing the machine learning model, app architecture, and how to set up and maintain the app.
- **API Documentation** if the app integrates with a server or third-party APIs for additional health data.

5. Testing & Quality Assurance

- Thorough testing of the app on multiple devices (unit tests, UI tests).
- **User Acceptance Testing (UAT)** to ensure the app meets user needs and works as expected.
- **Performance Testing** to ensure smooth operation on various Android devices.

Development Requirements

1. Technology Stack

- **Frontend:** Android development using **Kotlin** and **XML** for UI.
- **Machine Learning:** **TensorFlow Lite** for integrating the trained model into the mobile app.
- **Backend:** Laravel for backend management (for user data, model updates, etc.).
- **Push Notifications:** **Firebase Cloud Messaging (FCM)** for sending health-related notifications and alerts.
- **API Calls:** Retrofit or similar libraries to manage API calls for additional data or server sync.

2. Skills Required

- **Kotlin:** Expertise in Android development, especially for UI design and data management.
- **TensorFlow Lite:** Ability to convert machine learning models into TensorFlow Lite format and integrate them into Android apps.

- **Database Management:** Familiarity with **Room database** for local storage of health data.
- **Firestore Integration:** Knowledge of Firestore for user authentication, notifications, and analytics.
- **UI/UX Design:** Understanding of material design principles to create an intuitive and user-friendly app.
- **API Integration:** Experience in consuming external APIs (if needed) for fetching health-related data.

3. Resources

- **Blood Test Data:** Access to a dataset of blood test results to train and refine the machine learning model.
- **TensorFlow Model:** Pre-trained or custom-trained model for processing blood test results.
- **Medical Expert Collaboration:** Input from healthcare professionals to validate the accuracy and relevance of the health recommendations.
- **App Testing Devices:** Access to multiple Android devices for testing app performance and usability.

4. Timeline

- **Phase 1:** Project Setup & Initial Planning (1–2 weeks)
- **Phase 2:** Machine Learning Model Integration & Data Input Functionality (3–4 weeks)
- **Phase 3:** App Development (UI/UX, Local Storage, Notifications) (4–6 weeks)
- **Phase 4:** Testing, Debugging, & User Feedback (2–3 weeks)
- **Phase 5:** Final Adjustments, Deployment, and Launch (1–2 weeks)

5. App Deployment

- Deploy the app to the **Google Play Store** for public access or distribute it directly via APK for private distribution.
- Post-launch monitoring for user feedback and bug fixes.

2.3 Current System

IDC is an existing application used by healthcare providers to manage diagnostic tests and reports efficiently. It is designed to streamline the scheduling, processing, and storage of diagnostic test results, such as blood tests, imaging, and other lab reports. The system facilitates the tracking of patient records, generating diagnostic reports, and providing access to these reports for both patients and healthcare

professionals.

The application allows healthcare staff to input and manage patient data, schedule diagnostic tests, and automatically generate reports based on the results of the tests. Additionally, the system sends notifications to patients to inform them about their test results and any required follow-up actions. The primary goal of **IDC** is to provide a reliable method for managing and accessing diagnostic information quickly and accurately, making it a critical tool for hospitals and diagnostic centers.

Despite its effectiveness in handling test results and patient records, **IDC** lacks advanced features such as predictive analytics and personalized health recommendations. This gap is where the **EzHealth** system aims to offer an enhanced experience by providing personalized health insights based on the user's blood test data, offering a more proactive approach to health management.

Chapter 3

Requirement Analysis

3 Requirement Analysis

The requirement identifying technique for creating a use case diagram for EzHealth system involves stakeholder analysis, use case identification, use case description, use case relationships, and use case diagram construction.

Use Cases Diagram(s)

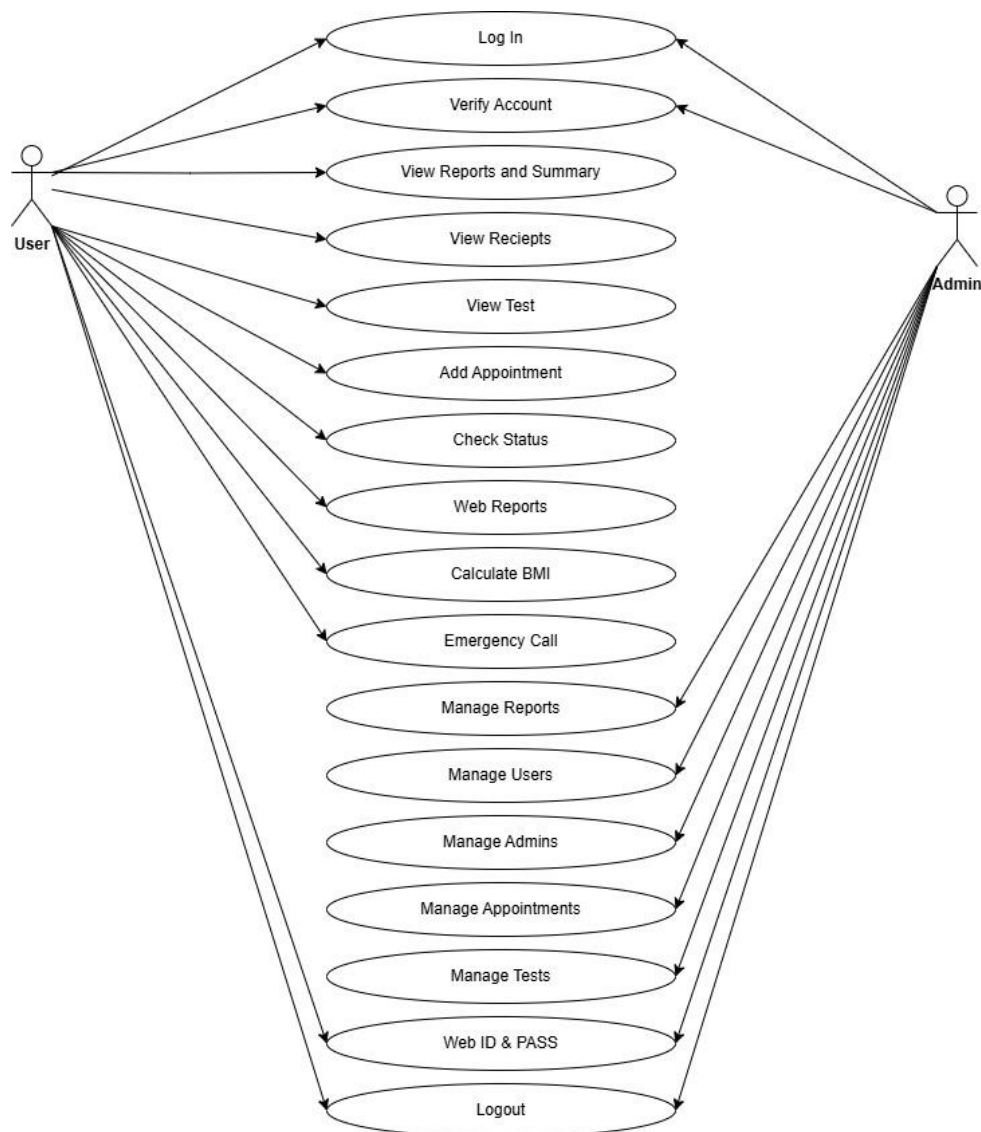


Figure 3.1. Use Case Diagram

Detailed Use Case

Use case descriptions provide a detailed narrative of how the system interacts with various actors to accomplish specific tasks or goals. Each use case outlines the interactions between users, administrators, and the EzHealth platform, capturing the functional requirements of the system. These descriptions serve as the foundation for understanding system behavior and ensuring all stakeholder needs are addressed during development. Use case descriptions bridge the gap between abstract requirements and tangible system designs. They provide clarity for developers, testers, and stakeholders.

3.1 Use Case Description

The table below indicates a comprehensive use case template filled in with an example drawn from the Online vehicle shipping management system.

Table 1. Login

Use case id	01
Use case name	Login
Actors	Admin , User
Description	Allows users and admins to log into the system.
Trigger	User/Admin requests to log in.
Pre-condition	The actor has an account.
Postcondition	The actor is now logged into the system.
Normal flow	This use case starts when an actor wants to login into the system. The system requests that the actor enter e-mail and password. The actor enters-mail and password. The system validates the entered e-mail and password and login into the system.
Alternative flow	For example, a Valid e-mail or an Invalid Password If in the basic flow the system cannot find the valid e-mail or the password is invalid, an error message is displayed.
Exceptions	Incorrect credentials. Request to re-enter the login details.
Business rules	The system must be connected to the network.

Table 2. Verify Accounts

Use case id	02
-------------	----

Use case name	Verify Account
Actors	• User • Admin
Description	Allows users and admins to verify account details for secure access.
Trigger	Verification request initiated.
Pre-condition	Account exists but is unverified.
Postcondition	Account is verified and marked as active.

Normal flow	This use case starts when the user is not logged in to the system and goes to the login page. The Actor requests verification The system sends verification link to that email The actor verifies.
Alternative flow	The link sent to user expires. The user requests new link
Exceptions	Verification failure due to network issues.
Business rules	Active internet access is available. Verification links expire after a set time

Table 3. *View reports and summary.*

Use case id	03
Use case name	View Reports and Summary
Actors	• User
Description	Allows users to view their medical reports and health summaries.
Trigger	User navigates to the reports section.
Pre-condition	User is logged in and has reports available.
Postcondition	User views the reports.
Normal flow	User selects reports System retrieves and displays reports.
Alternative flow	No reports found. Message displayed.
Exceptions	Server issues leading to delayed report retrieval.
Business rules	Only authenticated users can access reports.

Table 4. View Receipts

Use case id	04
Use case name	View Receipts
Actors	User
Description	Allows users to view their payment receipts..
Trigger	User navigates to the receipts section.
Precondition	User is logged in and has payment receipts available.
Postcondition	User views the payment receipts.
Normal flow	User selects receipts. System displays receipts.
Alternative flow	No receipts found. Message displayed.
Exceptions	There is no internet. Server down
Business rules	User must be authenticated.

Table 5. View Tests

Use case id	05
Use case name	View Tests
Actors	User
Description	Users can view a list of available tests.
Trigger	User navigates to the tests section.

Precondition	User is logged in and tests data is available.
Postcondition	List of tests is displayed to the user.
Normal flow	User selects the option System displays tests.
Alternative flow	No tests available Message shown.
Exceptions	There is no internet. An error occurs in the application. Server down
Business rules	Only logged-in users can view test details

Table 6. *Add Appointment*

Use case id	06
Use case name	Add Appointment
Actors	User
Description	Users can schedule appointments for tests or services.
Trigger	User initiates the add appointment process.
Pre-condition	The actors log in to the system.
Postcondition	Appointment is successfully added.
Normal flow	User enters details System saves appointment.
Alternative flow	Invalid details Prompt for corrections.
Exceptions	The error occurs in the application. The server is down.
Business rules	Appointment time slots must be available.

Table 7. *Check Status*

Use case id	07
Use case name	Check Status
Actors	User
Description	Users can check the status of their scheduled appointments.
Trigger	User accesses the appointment status section.

Precondition	User has scheduled appointments.
Postcondition	Appointment status is displayed.
Normal flow	User selects appointment. System shows status.
Alternative flow	No appointments. Notification shown.
Exceptions	There is no internet. Server down
Business rules	Only registered users can check status.

Table 8. Web Reports

Use case id	08
Use case name	Web Reports
Actors	User
Description	Users can access reports via the web.
Trigger	User navigates to web reports section.
Pre-condition	User is authenticated.
Postcondition	Reports are displayed in web view.
Normal flow	User enters cnic. User clicks. System retrieves reports.
Alternative flow	No reports found Notification shown.
Exceptions	Network issues.
Business rules	Only registered users have access.

Table 9. Calculate BMI

Use case id	09
Use case name	Calculate BMI
Actors	User
Description	Users can calculate BMI based on input data.
Trigger	User navigates to the BMI calculator.
Precondition	User is logged in..

Postcondition	BMI result is displayed.
Normal flow	User inputs weight/height. System calculates.
Alternative flow	Invalid input. Error prompt.
Exceptions	Calculation error due to invalid data.
Business rules	Input must meet required format.

Table 10. Emergency Call

Use case id	010
Use case name	Emergency Call
Actors	User
Description	Allows users to make an emergency call.
Trigger	User selects emergency call option.
Precondition	User is logged in.
Postcondition	User's phone redirects to dialer with active number.
Normal flow	User selects. Call initiated to emergency contact.
Alternative flow	Call not completed. Retry option shown.
Exceptions	No active number set or phone issues.
Business rules	Only valid contacts can be called.

Table 11. Manage Reports

Use case id	011
Use case name	Manage Reports
Actors	Admin
Description	Admins manage user reports, including review and upload.
Trigger	Admin accesses reports management.
Precondition	Admin is logged in.

Postcondition	Reports are updated, added, or reviewed.
Normal flow	Admin selects report System updates report status.
Alternative flow	Access denied due to permissions..
Exceptions	There is no internet. Database Update Error. Server down
Business rules	Only authorized admins can manage reports.

Table 12. Manage Users

Use case id	012
Use case name	Manage users
Actors	Admin
Description	Admins can create, update, or deactivate user accounts and verify user details for system access..
Trigger	Admin attempts to manage user data.
Precondition	Admin has logged in and has proper access rights.
Postcondition	User details are updated in the database and reflected in the system.
Normal flow	Admin successfully updates, creates, or deactivates a user account.
Alternative flow	Admin fails to modify user details due to invalid inputs or lack of permissions.
Exceptions	There is no internet. The error occurs in the application Server down
Business rules	Admin must validate user information and maintain data integrity while making updates.

Table 13. Manage Admins

Use case id	013
Use case name	Manage Admins
Actors	Super Admin
Description	Super Admin can create, modify, or deactivate admin accounts. Only the Super Admin can perform these actions.
Trigger	Super Admin initiates an action to manage admin details.
Precondition	Super Admin is logged in and has access to the admin panel..
Postcondition	Admin details are updated or new admin accounts are added to the system.
Normal flow	Super Admin successfully manages admin accounts..
Alternative flow	Super Admin fails to manage admin accounts due to missing information or application errors.

Exceptions	There is no internet. The error occurs in the application Server down
Business rules	Only the Super Admin has access to this module. Super Admin must ensure data integrity while making changes.

Table 14. Manage Appointments

Use case id	014
Use case name	Manage Appointments
Actors	Admin
Description	Admins can approve, reschedule, or cancel user appointments for consultations or sample collections.
Trigger	Admin receives appointment requests from users.
Precondition	Appointment requests are received and logged in the system...
Postcondition	Users are notified about the status of their appointments.
Normal flow	Admin successfully updates appointment statuses (approved, rescheduled, or canceled)
Alternative flow	Admin fails to update appointment statuses due to missing or incorrect information.
Exceptions	There is no internet. The error occurs in the application Server down
Business rules	Appointment records must be updated promptly and accurately

Table 15. Manage Tests

Use case id	015
Use case name	Manage Tests
Actors	Admin
Description	Admins can update the list of available medical tests, including their descriptions, costs, and requirements.
Trigger	Admin attempts to add, edit, or delete test details..
Precondition	Admin is logged in, and the system has existing test categories.
Postcondition	Updated test details are displayed to users on the platform.
Normal flow	Admin successfully updates test details.
Alternative flow	Admin fails to modify test details due to invalid input or system errors.
Exceptions	There is no internet. The error occurs in the application Server down
Business rules	Admin must ensure all test data is accurate and categorized correctly.

Table 16. Web ID and Password

Use case id	016
Use case name	Web ID and Password
Actors	User, Admin

Description	Allows users and admins to use Web ID and password for login.
Trigger	User/admin enters Web ID and password
Precondition	Web ID and password are set up and valid.
Postcondition	Successful login using Web ID.
Normal flow	Enter Web ID and password System validates Access granted.
Alternative flow	Invalid Web ID/password Error message.
Exceptions	There is no internet. The error occurs in the application Server down
Business rules	Strong password enforcement required.

Table 17. Logout

Use case id	017
Use case name	Logout
Actors	User, Admin
Description	Allows users and admins to securely log out of the system.
Trigger	User/admin selects the logout option.
Precondition	User/admin is logged in.
Postcondition	User/admin is logged out.
Normal flow	User/admin clicks logout Session ends.
Alternative flow	Logout fails due to server error Retry prompt.
Exceptions	Session timeout occurs.
Business rules	Logout confirmation is displayed.

3.2 Functional Requirements

The EzHealth mobile application is designed to provide personalized health recommendations based on blood test data. Below are the key functional requirements that define the behavior of the system

FR1 User Registration and Login

The system shall allow users to create an account using email and password, and log in to securely access personalized features.

FR2 Blood Test Data Input

The system shall provide an interface for users to manually input their blood test values, such as glucose, cholesterol, and hemoglobin levels.

FR3 Machine Learning-Based Health Prediction

The system shall process the user's blood test data using an integrated TensorFlow Lite model and generate relevant health predictions or alerts.

FR4 Display of Health Recommendations

The system shall display personalized health recommendations to users based on the output of the prediction model. These recommendations may include dietary advice, risk assessments, and follow-up suggestions.

FR5 Local Storage of Test History

The system shall store a history of previously entered blood test data and corresponding recommendations locally using Room Database for offline access.

FR6 User Profile Management

The system shall allow users to update personal profile information such as name, age, and gender to help tailor recommendations.

FR7 Notifications and Alerts

The system shall send notifications to users for health check reminders, abnormal test result alerts, and general wellness tips.

FR8 Admin Panel Integration

The system shall integrate with an existing Laravel-based admin panel for managing user data, updating the model, and overseeing health recommendation guidelines.

FR9 Data Synchronization

The system shall synchronize locally stored data with the server when an internet connection is available to ensure backup and consistency.

FR10 Graphical Representation of Results

The system shall visualize the blood test trends using graphs and charts to help users better understand changes in their health over time.

3.3 Non-Functional Requirements

NFR1 Usability

The system shall provide a user-friendly and intuitive interface that can be easily navigated by users with minimal technical knowledge.

NFR2 Performance

The application shall process blood test data and return health recommendations

within a few seconds to ensure a smooth user experience.

NFR3 Availability

The mobile application shall be available for use 24/7, with offline access to previously saved data using local storage (Room Database).

NFR4 Reliability

The system shall consistently provide accurate predictions and recommendations, based on the trained machine learning model, without failure under normal conditions.

NFR5 Scalability

The system shall be designed in a way that it can handle an increasing number of users and data entries without significant performance degradation.

NFR6 Maintainability

The application code shall be modular and well-documented to allow future updates, such as model upgrades or UI improvements, with minimal effort.

NFR7 Security

The system shall ensure that all user data, including personal and medical information, is securely stored and transmitted using encryption protocols.

NFR8 Compatibility

The application shall be compatible with Android devices running Android 8.0 (Oreo) and above.

NFR9 Portability

The system design shall allow for potential future development of an iOS version without major architectural changes.

NFR10 Data Integrity

The system shall ensure that user data is not lost or corrupted during input, storage, or synchronization with the server

Chapter 4

Design and Architecture

4 Design and Architecture

The design and architecture of the EzHealth system focus on delivering a lightweight, intelligent, and user-friendly mobile healthcare solution. The core objective is to empower users to monitor their health using personalized medical insights derived from blood test data, even in offline conditions.

The system adopts a modular design that separates the user interface, machine learning logic, data management, and backend operations. This layered approach promotes scalability, maintainability, and ease of integration with future enhancements or third-party services.

EzHealth uses a **client-side architecture** centered around an Android mobile application, which integrates a **TensorFlow Lite** model for on-device health prediction. A **local Room database** is used to store user input and results for offline access. For extended management and control, an **admin panel built with Laravel** is connected to the app via secure API endpoints, allowing remote monitoring, model updates, and content management.

The design follows modern mobile development standards using **Kotlin and XML**, ensuring a responsive user interface and optimized performance. Firebase services are employed for authentication and push notifications, enhancing the app's connectivity and interactivity.

The architecture is optimized for data privacy, offline usability, and low-latency health prediction, making it suitable for real-world use, especially in regions with limited internet access.

4.1 System Architecture

The EzHealth system is based on a client-server architecture with integrated machine learning functionality. The main component is the Android mobile application, which interacts with a TensorFlow Lite model, local storage, and optionally a backend admin panel for extended functionality.

The system is divided into the following major modules:

The Mobile Application is the core interface for users. It allows users to input their blood test data, view health recommendations generated by the integrated TensorFlow Lite model, and review historical test records stored locally using Room database. The application also supports offline functionality and syncs data with the backend server when an internet connection is available.

The Machine Learning Model, converted into TensorFlow Lite format, runs directly on the user's device without needing a constant internet connection. This model processes the blood test input and provides health risk predictions and personalized recommendations.

The Laravel Admin Panel is used by administrators to manage user data, monitor system usage, and deploy updates to the machine learning model or health guidelines. The admin panel can also store data backups and provide analytics for system performance and user behavior.

The Firebase Services handle user authentication, push notifications, and optional cloud data storage or syncing.

This architecture ensures high performance, offline availability, and secure handling of sensitive health data. It also allows the system to scale by updating models and managing users via the backend.

4.2 Data Representation [Diagram + Description]

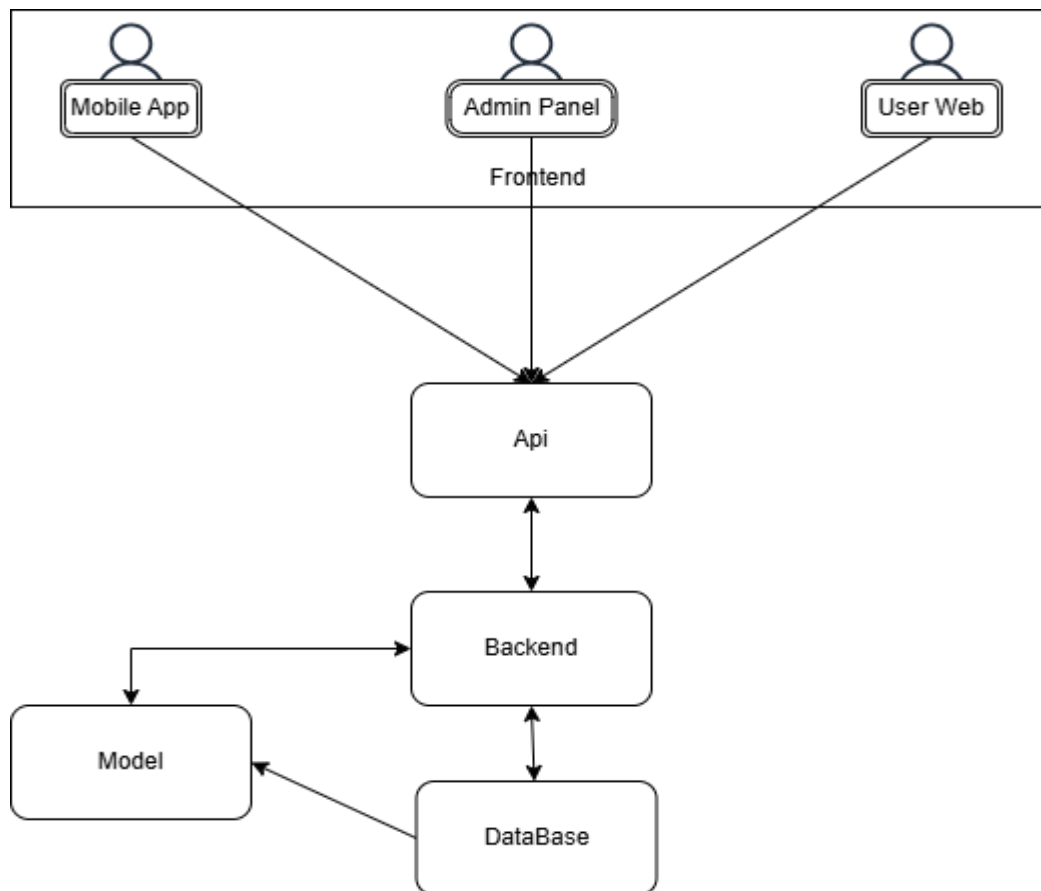


Figure 4.1: Architecture Diagram

Ez Health project consists of a mobile app, admin panel, and web panel as the front-end components, all connected to the back-end through APIs. The back-end is integrated with Firebase as the database for data storage and real-time synchronization. The system is designed with a model layer that processes business logic, ensuring seamless communication between the front-end and back-end. This architecture supports scalability, flexibility, and ease of maintenance for the overall project.

4.3 Process Flow/Representation

This flowchart provides a detailed overview of the interactions within the system, starting from user registration to the termination of the session. It highlights decision points like email and account verification, ensuring secure access for users. Additionally, the admin section supports efficient management of system operations, enhancing overall functionality and user experience. The workflow

ensures streamlined navigation for both users and administrators.

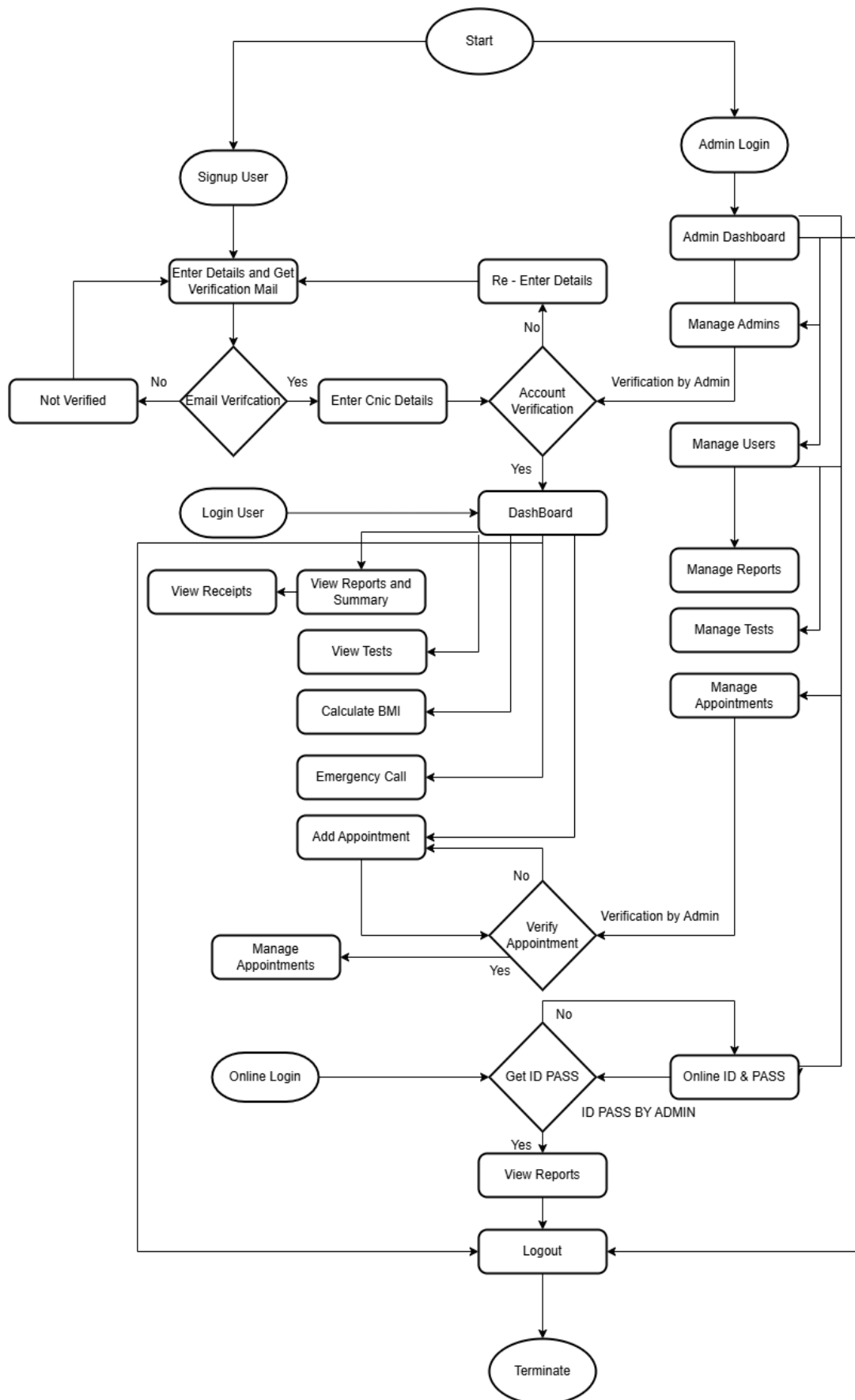


Figure 4.2: Flow Diagram

Design Models [along with descriptions]

This is important to include design models that provide a clear and comprehensive understanding of the system's structure and functionality.

4.4 Sequence Diagram

The sequence diagram for Ez Health shows the steps a user takes when booking an appointment, accessing medical test results, or receiving AI-generated summaries of reports. After booking, the user is notified of appointment confirmations or changes, while administrators verify the user's information. Medical records and test results are retrieved and displayed with AI summaries for easy understanding. This is the following Sequence Diagram of our project.

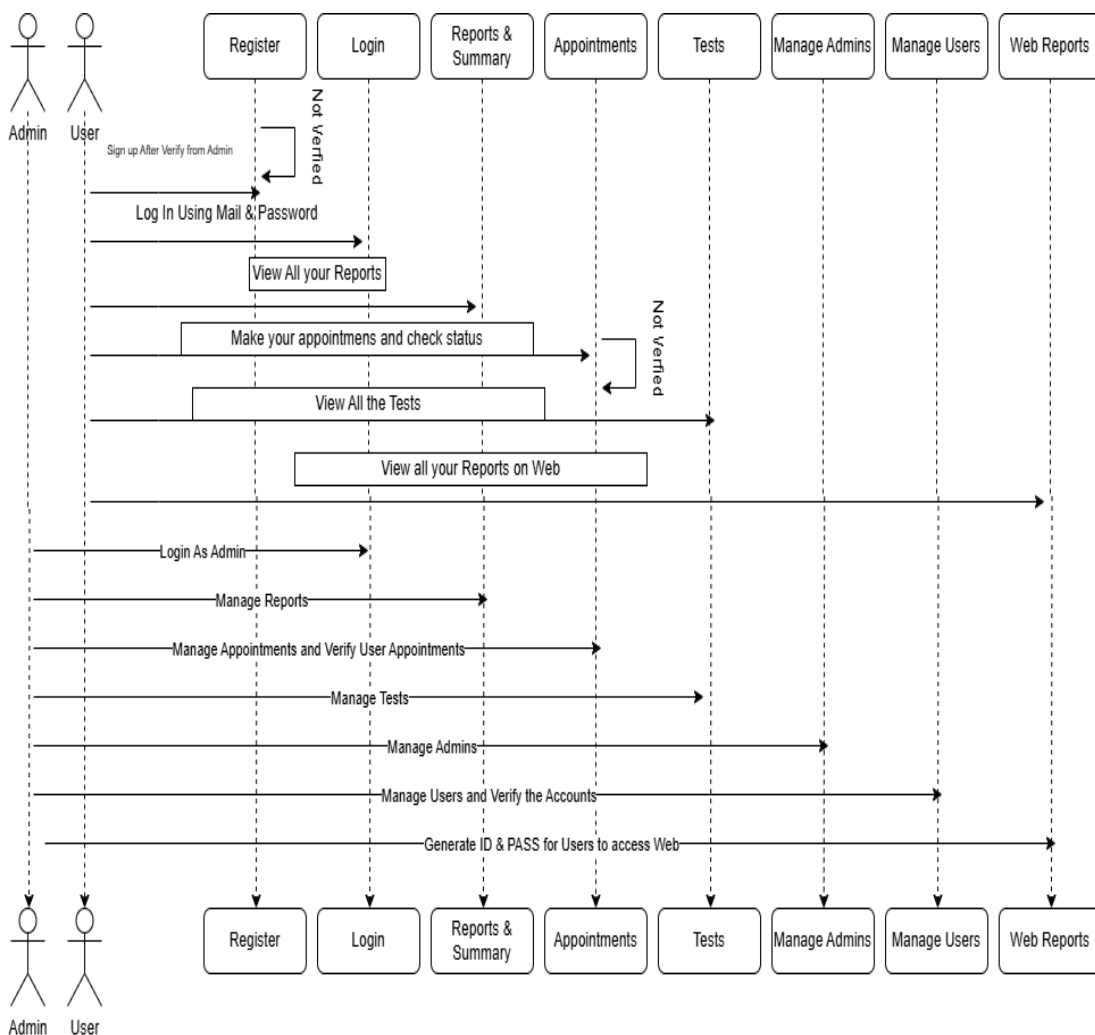


Figure 4.3: Sequence Diagram

Class Diagram

The attributes and methods of each class can be further defined based on your specific requirements and implementation details in the class diagram.

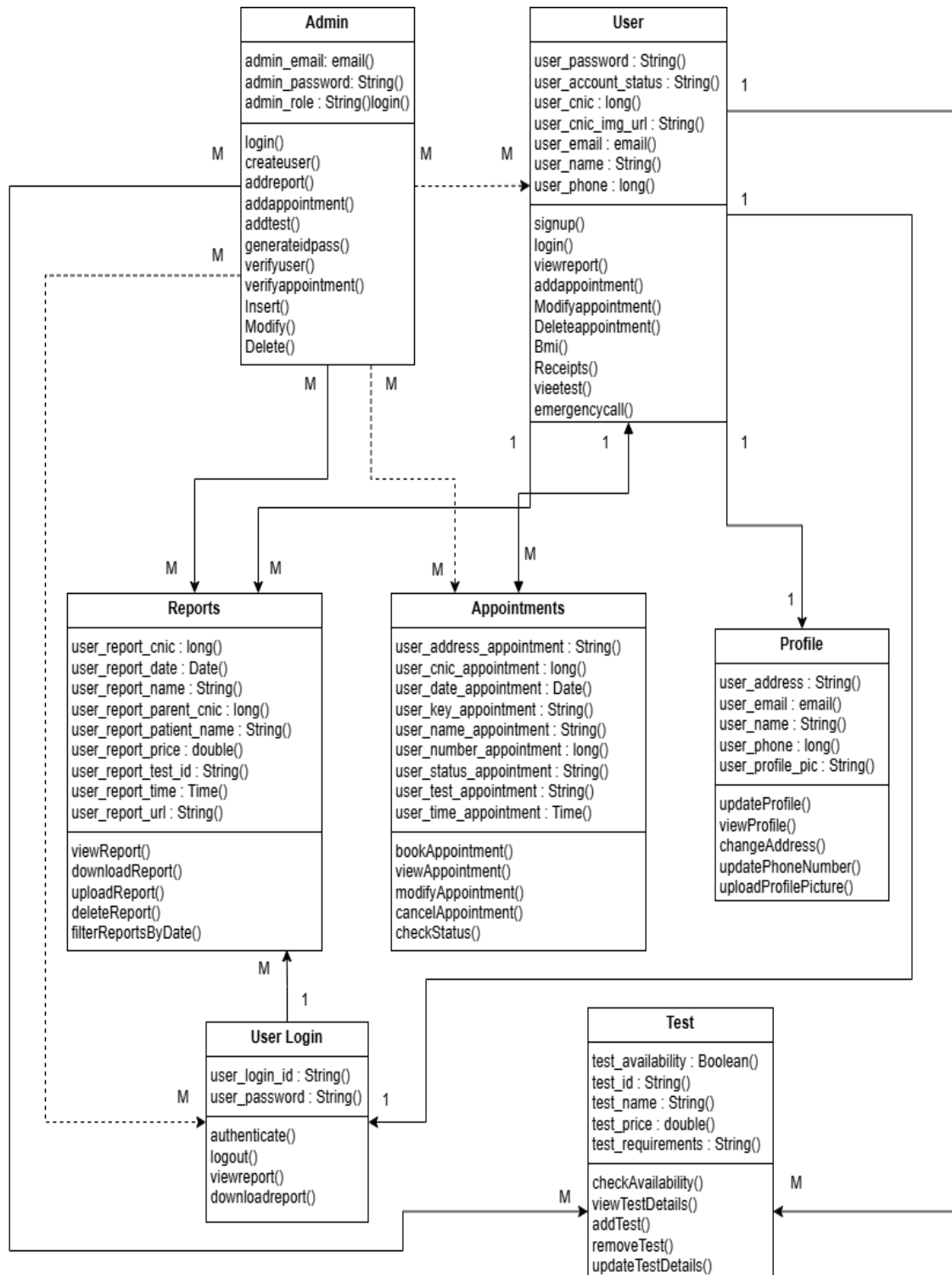


Figure 4.4: Class Diagram

Chapter 5

Implementation

5 Implementation

The UML Class Diagram outlines a robust system designed for healthcare or appointment management, featuring distinct roles for admins and users. The 'Admin' class serves as the backbone for system oversight, equipped with attributes like 'admin_email' and 'admin_role' to define administrative identity and permissions. Methods such as 'createreport()', 'addappointment()', and 'generatereport()' enable admins to manage critical operations, including creating and verifying reports and scheduling appointments. The 1-to-M relationships with 'Reports', 'Appointments', and 'Test' classes indicate that a single admin can oversee multiple instances of these entities, ensuring centralized control over system data and workflows.

The 'User' class caters to end-users, likely patients, with attributes like 'user_cnic', 'user_email', and 'user_phone' for identification and communication. Users can perform self-service tasks through methods such as 'signup()', 'login()', and 'addappointment()', allowing them to register, authenticate, and book appointments. Additional functionalities like 'Bmi()' for health calculations and 'Receipts()' for transaction records enhance user experience. The diagram's 1-to-M relationships with 'Appointments' and 'Reports', along with a 1-to-1 relationship with 'Profile' and 'emergencycall', suggest a personalized and secure user management system where each user can manage multiple appointments and maintain a single emergency contact.

The 'Reports' and 'Appointments' classes are pivotal for data management within the system. The 'Reports' class includes attributes like 'user_report_date', 'user_report_price', and 'user_report_url', supporting detailed record-keeping, while methods such as 'viewReport()', 'downloadReport()', and 'filterReportsByDate()' provide users with flexible access and organization options. Similarly, the 'Appointments' class tracks details like 'user_date_appointment' and 'user_status_appointment', with methods like 'bookAppointment()', 'modifyAppointment()', and 'checkStatus()' enabling dynamic scheduling and status updates. The M-to-1 relationships with the 'User' class ensure that multiple reports and appointments are linked to individual users,

facilitating personalized healthcare tracking.

The `'Profile'` and `'User_Login'` classes focus on user identity and access control. The `'Profile'` class, with attributes like `'user_address'` and `'user_profile_pic'`, allows users to maintain and update personal information through methods such as `'updateProfile()'` and `'updateProfilePicture()'`. The 1-to-1 relationship with the `'User'` class ensures each user has a unique profile. Meanwhile, the `'User_Login'` class, featuring `'user_login_id'` and `'user_password'`, handles authentication via the `'authenticate()'` method, with a 1-to-1 relationship to the `'User'` class, ensuring secure and individualized login sessions that support actions like `'viewreport()'` post-authentication.

Finally, the `'Test'` class supports diagnostic or medical testing functionalities, with attributes like `'test_available'` and `'test_price'` to manage test status and cost. Methods such as `'checkAvailability()'` and `'addtest()'` allow for test management and updates, with M-to-1 relationships to `'User'` and `'Appointments'` classes indicating that tests are linked to specific users and appointments. This modular design supports scalability and security, with potential for future enhancements like role-based access control, emergency notifications, and advanced filtering, making the system adaptable to evolving needs in healthcare management.

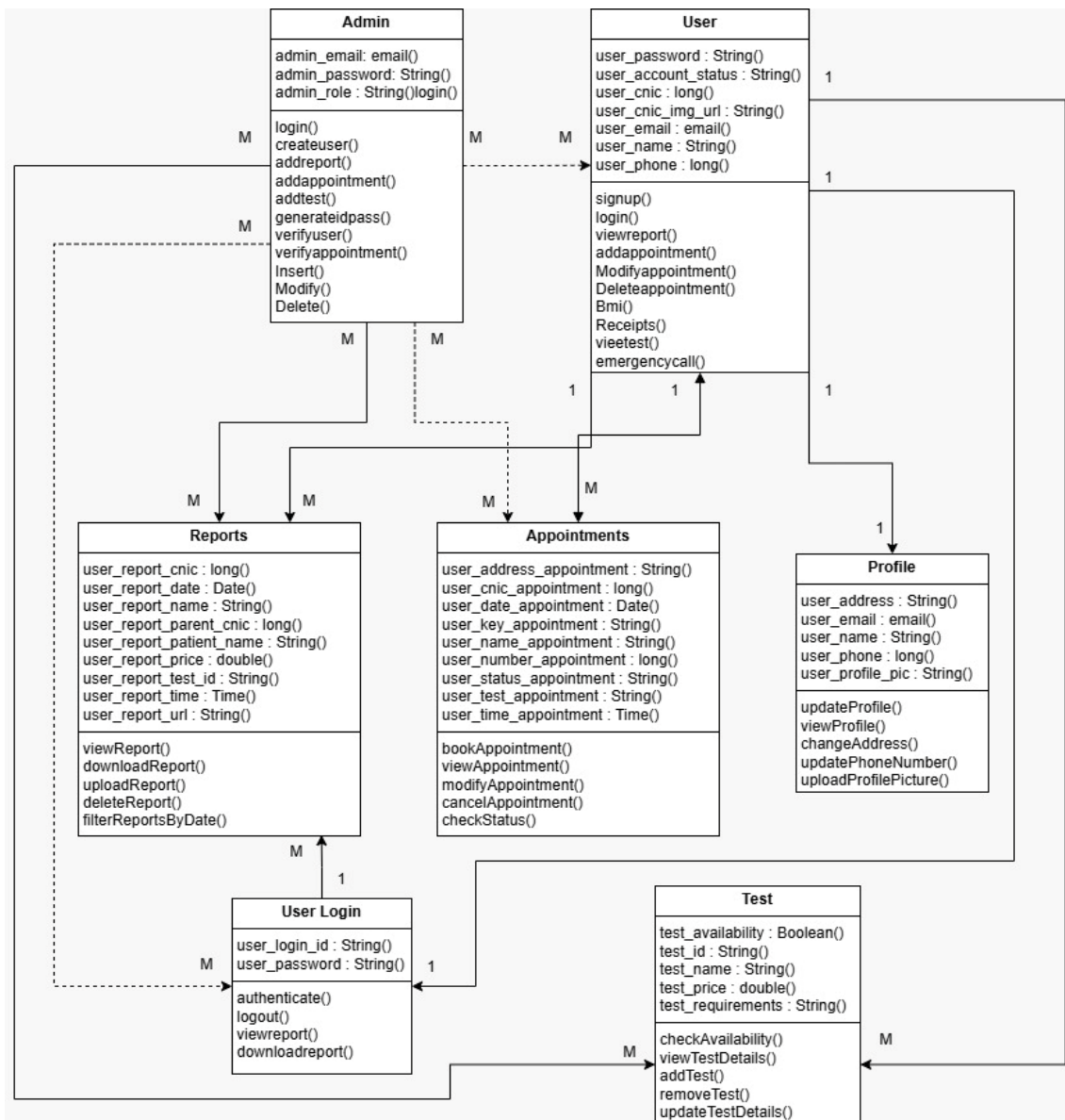


Figure 5.1: UML Diagram

5.1 Algorithm

EzHealth utilizes a machine learning-based prediction mechanism integrated within the mobile application using TensorFlow Lite. The primary algorithm at the core of the system is a trained classification model, such as a Decision Tree or Neural Network, which processes user-input blood test data to predict health conditions. When a user enters their blood test values, these inputs are formatted into the required structure and passed to the TensorFlow Lite model. The model evaluates the data and returns a classification output, which is mapped to a specific health condition. The app then displays a corresponding recommendation,

such as "Normal", "At Risk", or "Critical", based on the result.

In addition to prediction, the system incorporates data persistence using local storage via Room Database. When a prediction is generated, the input values along with the prediction and a timestamp are saved locally. This allows users to access their health records offline and track their health trends over time. Users can also retrieve their history through simple querying and sorting mechanisms.

The Laravel-based admin panel operates through RESTful APIs and handles administrative functions like viewing user activity, updating health guidelines, and uploading new versions of the machine learning model. When a new model is uploaded to the server, the app is notified (via Firebase or manual sync) and can update its model version accordingly.

5.2 Pseudo Code

Algorithm PredictHealthCondition(Input: BloodTestValues)

 Load TFLite_Model

 Preprocess Input to Match Model Format

 Output $\leftarrow$ TFLite_Model.Predict(Input)

 if Output == 0 then

 Return "Normal"

 else if Output == 1 then

 Return "At Risk - Lifestyle Changes Recommended"

 else

 Return "Critical - Immediate Medical Attention Advised"

 end if

End Algorithm

These algorithms and workflows collectively ensure that EzHealth provides a responsive, intelligent, and privacy-focused health monitoring experience.

5.3 External APIs

Describe the APIs used in the following table.

Table 18. Details of APIs used in the project

Name of API	Description of API	Purpose of usage	List down the function/class name in which it is used
Gemini API	Gemini is a multimodal generative AI API	It is used in EzHealth to implement an AI-	GeminiChatBotManager, handleUserQuery(), generateHealthResponse()

	developed by Google that supports text-based input and output for natural language understanding and generation.	powered health chatbot that can respond to user queries related to symptoms, general health advice, and interpretation of test results.	
Google ML Kit (Text Recognition)	ML Kit is a mobile SDK that brings Google's machine learning capabilities to Android. The Text Recognition API extracts text from images in real time or static input.	It is used to parse blood test reports captured via the mobile camera or uploaded images and automatically extract relevant health values (e.g., glucose, cholesterol).	ReportImageProcessor, extractTextFromImage(), parseHealthReportData()

5.4 User Interface

The user interface of the EzHealth mobile application is designed to be clean, minimal, and user-friendly, keeping accessibility and simplicity in mind. The app provides users with intuitive navigation and clearly labeled options for interacting with the core features, including report-based health suggestions and an AI-powered chatbot.

The main screens and their descriptions are as follows

Dashboard Screen

The dashboard serves as the central hub for users, displaying key information such as health stats, upcoming appointments, and quick access options. It offers a clean and intuitive layout for easy navigation. It ensures that users have quick access to essential features, keeping their health journey streamlined and efficient. The dashboard also provides real-time updates for a personalized experience.

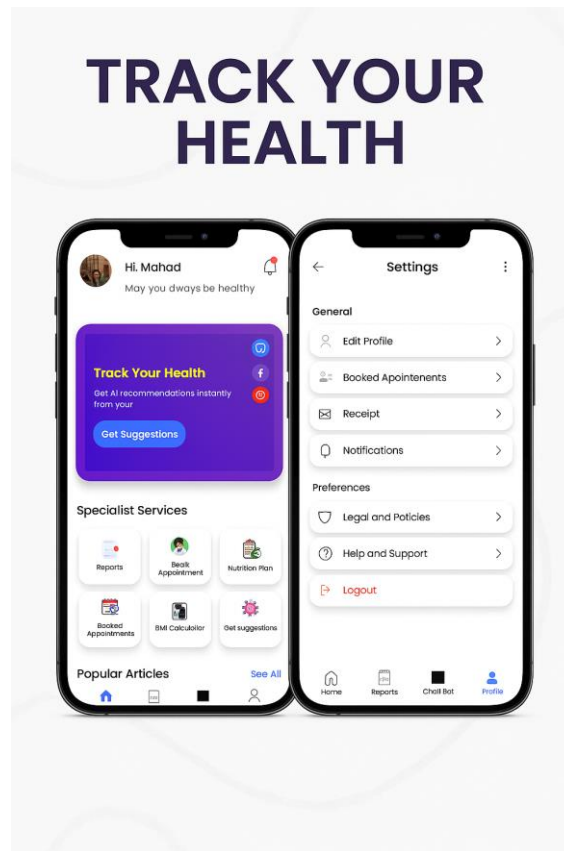


Figure 5.2:DashBoard Screen

Booked Appointments Screen

This screen lists all upcoming and past appointments, complete with details like date, time, and status. Users can easily manage or cancel their bookings from here. Users also receive notifications for upcoming appointments, ensuring they never miss a scheduled test. They can reschedule or cancel appointments directly from this screen.

← Book Appointment ⋮

🕒 12:00 ▼

Selected Time: None

🩺 Enter Medical Issues

📍 Enter Address

Book Appointment

Figure 5.3 :Appointments Screen

Body Mass Index Screen

The BMI screen allows users to calculate their body mass index by entering height and weight. It provides an instant result along with health tips based on the BMI range. The visually appealing interface makes health tracking both engaging and straightforward for users.

2:29 AM 60%

esqhv3xjm.streamlit.app

Fork

User Demographics & Activity Level

Enter your age:

25 - +

Select your gender:

Male ▾

Enter your height (in cm):

170 - +

Enter your weight (in kg):

70 - +

Select your activity level:

Sedentary ▾

Select your goal:

Maintain Weight  

Figure 5.4: BMI Screen

Reports Screen

This screen showcases medical reports in an organized format, allowing users to view, download, or share them conveniently. Each report is categorized for easy access. Users can also search or filter reports by date, making it easier to find specific health records. Additionally, the screen allows for downloading or sharing reports securely.

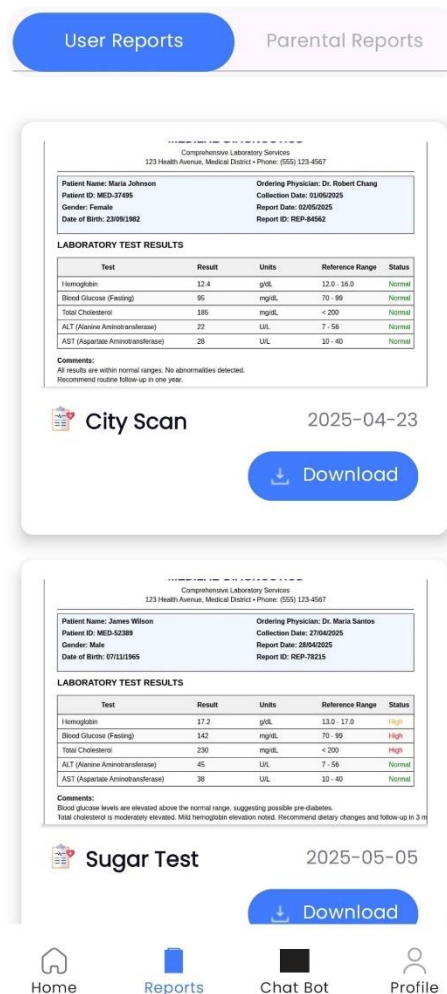


Figure 5.5 : Report Screen

Health Chatbot Screen

This screen allows users to interact with an AI-based health assistant powered by the Gemini API. The user can type in queries about symptoms, medications, lifestyle tips, or general health concerns. The chatbot provides intelligent, real-time responses based on the user's input, creating a natural and informative conversational experience. The chat interface resembles a messaging app, with clearly separated user messages and bot replies. This feature enhances user engagement and offers accessible health advice without the need for clinical appointments.

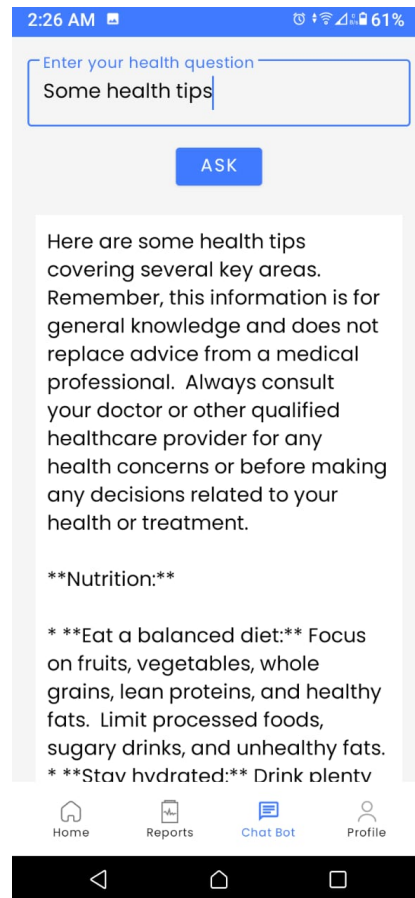


Figure 5.5: ChatBot

Report Selection and Suggestion Screen

Once the report data is uploaded, users can view a list of previously stored reports in a structured format. By selecting any report, the app displays detailed health analysis including detected conditions and recommended actions. These suggestions include tailored diet plans, exercise routines, and preventive care based on the report's parameters and model prediction. The design focuses on readability with highlighted risk levels and expandable cards for each suggestion. This empowers users to take proactive steps toward better health based on personalized insights.

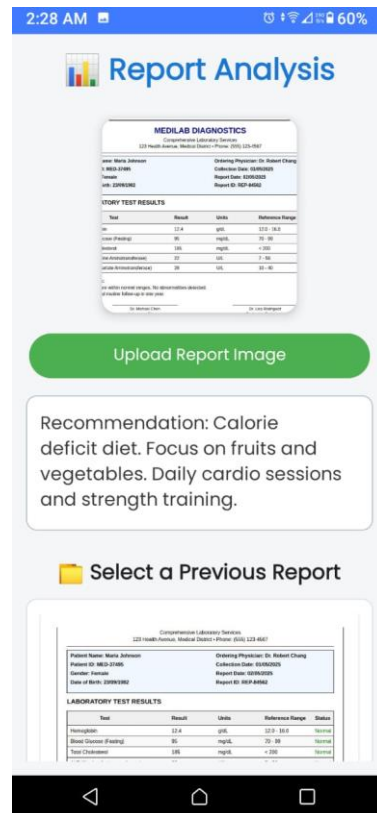


Figure 5.7: Report Analysis

Booked Appointment Screen

The "Booked Appointments" screen allows users to view all their previously scheduled appointments in a clean, list-based layout, fetched dynamically from the Firebase Realtime Database. Each appointment entry displays relevant details such as date, time slot, and doctor or department (if applicable). To enhance usability, the screen supports swipe-to-delete functionality, enabling users to remove any appointment by simply swiping left or right. Upon swiping, a confirmation dialog ensures the action is intentional, and once confirmed, the appointment is deleted from both the interface and the database. This feature provides users with full control over their bookings while keeping the interface responsive and intuitive.

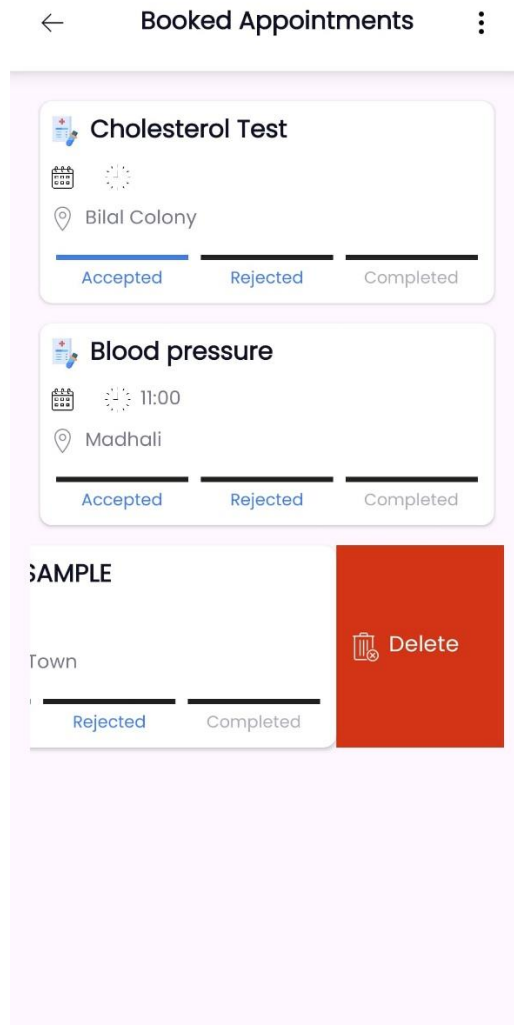


Figure 5.8: Booked Appointment Screen

Admin Dashboard Screen

The admin panel's dashboard acts as the central control hub, offering a summarized view of all key system components. It displays total registered users, uploaded medical reports, currently booked appointments, and the list of available diagnostic tests. This provides the admin with real-time insights into the system's overall usage and activity. The interface is designed with clear cards or tiles for each category, making it easy to navigate and monitor the application's backend operations at a glance.

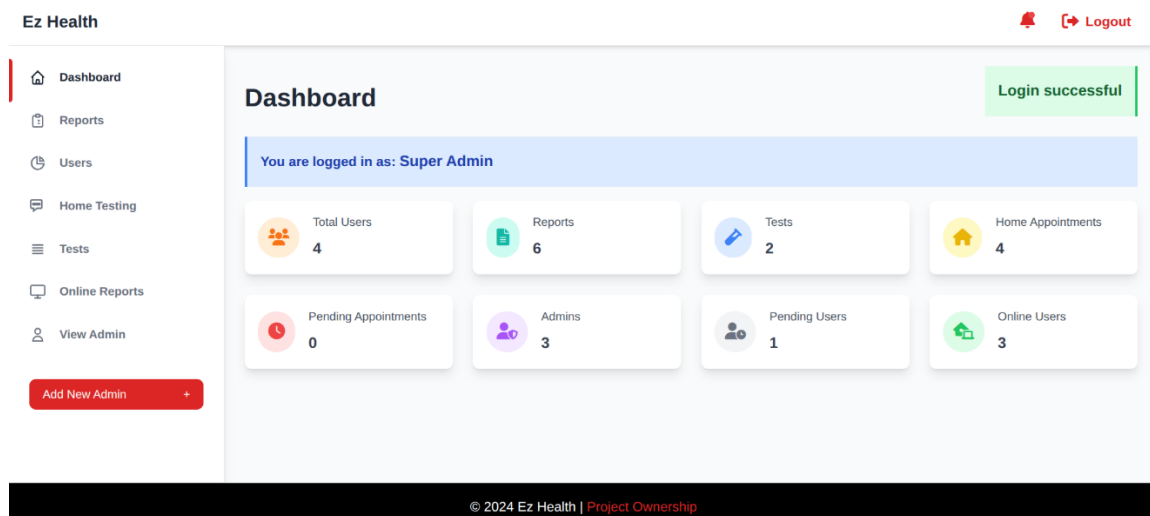


Figure 5.9: Admin Dashboard

Admin Report Upload Screen

One of the core features of the admin panel is the Report Upload Activity, which enables administrators to upload medical reports directly to a specific user's profile. The admin selects a user based on their CNIC or email, chooses the report file (usually in image or PDF format), and adds relevant metadata such as test name, date, or notes. Once uploaded, the report is stored in Firebase Storage, and its reference is saved under the user's record in the Firebase Realtime Database. This ensures secure access and linkage of medical records

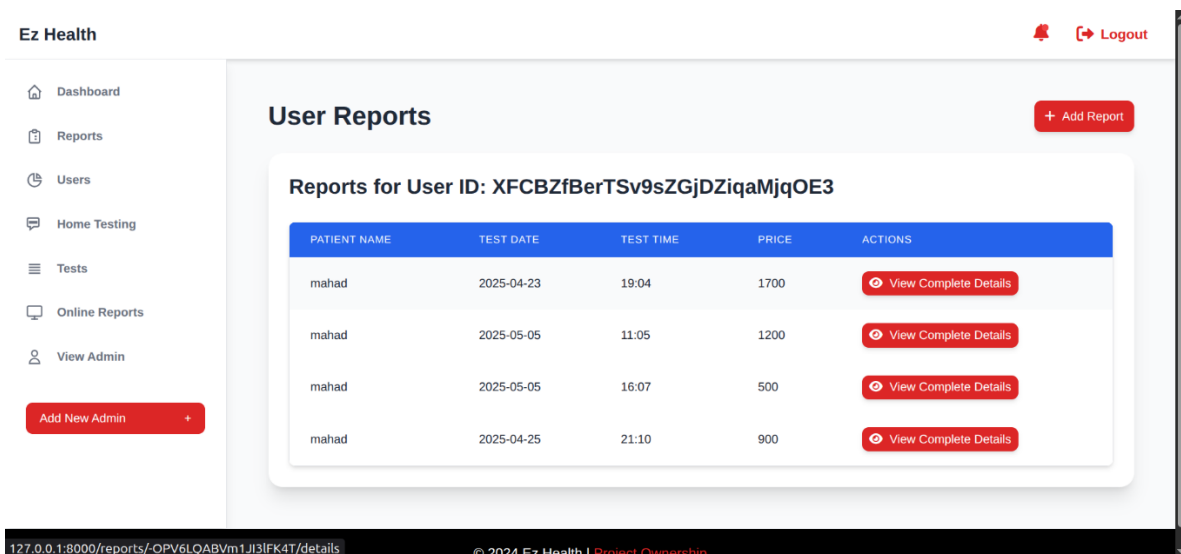


Figure 5.10 : Admin Report Screen

Admin Login

The admin panel features a secure login system, allowing only authorized personnel to access backend functionalities. Among the admin roles, there is a designated **Super Admin** who holds elevated privileges. The Super Admin can create new admin accounts, assign roles, and remove or deactivate existing admins when necessary. This hierarchical access control ensures that sensitive operations, such as user and system configuration management, are restricted to trusted personnel, thereby maintaining the integrity and security of the platform.

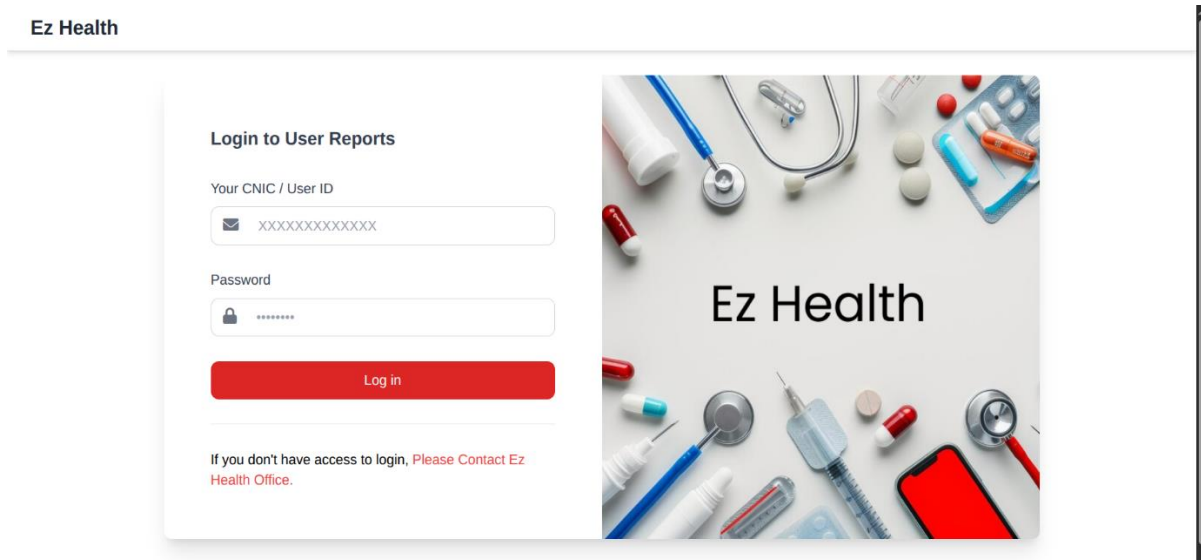


Figure 5.11: Admin Login Screen

Report Details Screen

The Report Details Activity provides a comprehensive view of individual medical reports stored in the system. Each report entry displays critical information such as the date of the test, the report generation time, and the associated price of the medical service. It also includes the patient's name and CNIC, ensuring clear identification and traceability of the report. This feature allows admins to quickly verify report data, cross-check with appointments or payments, and assist users in case of any queries or discrepancies.

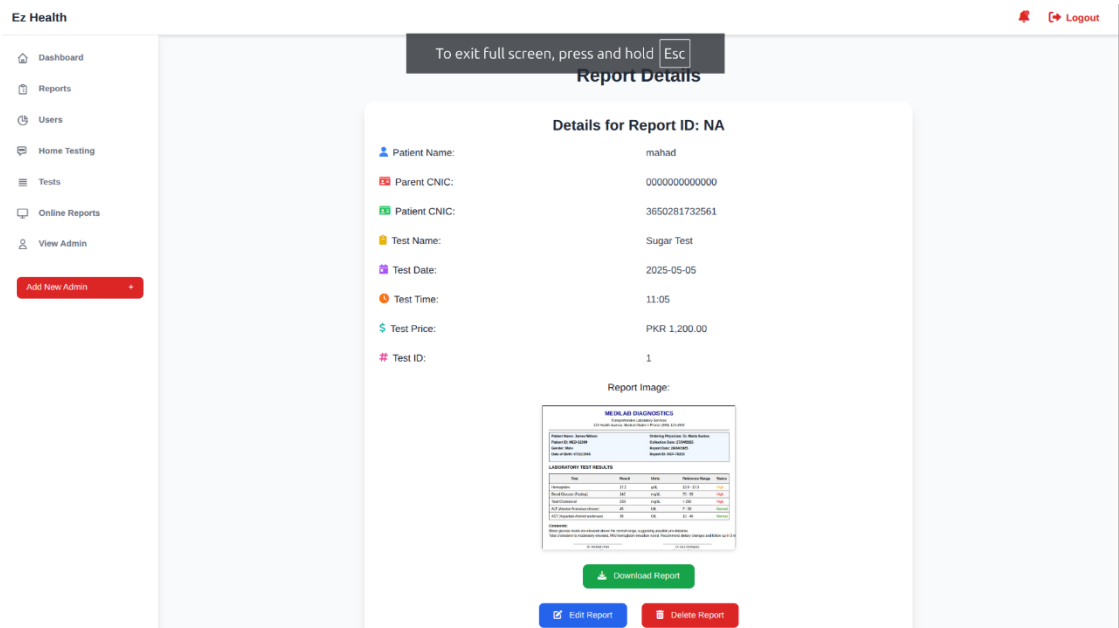


Figure 5.12 : Report Details Screen

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

To ensure the correctness, reliability, and performance of the EzHealth application, both manual and automated testing approaches were adopted. Testing was conducted at the module level as well as at the system integration level to validate functionality, usability, and performance. The results of these tests confirmed that the application meets the defined functional and non-functional requirements.

6.1 Manual Testing

Manual testing involved step-by-step execution of user actions to identify any functional issues, design inconsistencies, or usability problems. Each screen and feature of the application was manually tested on real Android devices to ensure correct behavior under normal and edge-case scenarios.

6.2 System testing

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. There are few types of testing which includes the unit testing, functional testing and integration testing. The testing must be completed before it is being deploy for user to use.

6.3 Unit Testing

System testing ensures that the **EzHealth** app works as intended and meets the requirements specified earlier in the project. This phase involves testing individual modules as well as their interaction within the system. The primary goal of system testing is to identify hidden errors and ensure that the system behaves as expected in different scenarios.

Unit Testing 1: Login as FYP Committee

Testing Objective: To ensure the login form is working correctly

Table 19. Unit Testing 1

No.	Test case/Test script	Attribute value	and	Expected result	Result

1.	Verify user login after click on the „Login“ button on login form with correct input data	Username: mahad7341b@gmail.com Password: 12345678	Successfully log into the main page of the system as FYP Committee member.	Pass
2.				

Unit Testing 2: Edit Profile

Testing Objective: To ensure the edit profile form is working properly.

Table 20. Unit Testing 2

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Click Edit Profile and change profile picture	Upload new profile picture	Profile picture will be changed and new profile picture will be shown in profile and home page	Pass

Unit Testing 3: Health ChatBot

Testing Objective: To ensure the health chatbot is working correctly.

Table 21. Unit Testing 3

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Ask Health related questions about any disease	Disease name	It will tell details about disease	Pass

6.4 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each of the module is tested. This is to ensure that the system produced meets the specifications and requirements.

Functional Testing 1: Login with different roles

Objective: To ensure that the correct page with the correct navigation bar is loaded.

Table 22. Functional Testing 1

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as a „FYP Committee“ member.	Username: mahad7341b@gmail.com Password: 12345678	Main page for the FYP Committee member is loaded with the FYP Committee navigation bar	Pass
2.				

Functional Testing 2: Health Chatbot Response

Objective: To ensure that the health chatbot responds correctly to user queries.

Table 23. Functional Testing 2

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Ask the chatbot for suggestions for managing high blood pressure	Query: “How to manage high blood pressure?”	The chatbot responds with general advice on managing hypertension.	Pass
2.				

Integration Testing

Table 24. Integration Testing

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as a user and interact with the app	Username: mahad7341b@gmail.com Password: 12345678	Successfully log in and access the health app	Pass
2.	Upload a report and receive health suggestions	Report Image: Blood test result	Upload the report, and the health suggestions are displayed (e.g., diet and exercise recommendations).	Pass
3.	View historical reports and suggestions	Historical Report: Previously uploaded data	Display a list of previously uploaded reports and associated health suggestions.	Pass

6.5 Automated Testing:

Tools used:

Table 25. Automated Testing

Tool Name	Tool Description	Applied on [list of related test cases / FR / NFR]	Results
JUnit	A widely used testing framework for unit testing Java/Kotlin code	Applied on unit tests for report upload and prediction logic (e.g., <code>parseHealthReportData()</code> , <code>generateHealthResponse()</code>)	Pass
Mockito	A mocking framework used for unit testing and simulating dependencies	Applied on functions that interact with external services (e.g., Gemini API, Firebase)	Pass
Espresso	A UI testing tool for Android to simulate user interaction and validate UI behavior	Applied on tests for uploading reports, receiving health suggestions, and interacting with the chatbot	Pass

Chapter 7

Conclusion

7 Conclusion and Future Work

This chapter concludes the project and highlights future work.

7.1 Conclusion

The EzHealth project was developed to offer a smart, mobile-based health assistant that helps users understand and improve their health based on real medical report data. Using advanced technologies such as the Gemini API for chatbot integration and Google ML Kit for image-based report parsing, the application successfully delivers health recommendations including diet and exercise plans tailored to individual test results. The user-friendly interface and interactive design ensure that users can easily upload reports, interact with the chatbot, and access suggestions with minimal effort.

The system was tested thoroughly through both manual and automated methods, and the results confirm that it meets the functional and non-functional requirements. The Laravel-based admin panel supports secure backend operations, ensuring that data handling remains efficient and reliable. Overall, EzHealth meets its goal of providing intelligent, personalized health support in an accessible mobile platform.

7.2 Future Work

To enhance the functionality and impact of EzHealth, several improvements and expansions are planned for the future. One key area is the integration of additional machine learning models and medical APIs to offer broader diagnostics and predictive healthcare insights. Voice assistant features and multilingual support can improve usability for users with different needs and backgrounds.

Another promising direction is to connect EzHealth with wearable health devices such as smartwatches and fitness trackers, enabling real-time monitoring and more dynamic health suggestions. Moreover, implementing stronger data encryption, user consent mechanisms, and ensuring compliance with healthcare data protection regulations like HIPAA will be critical for scaling the app to a larger audience. These advancements will make EzHealth a more comprehensive and secure digital health companion.

References

[1] G. Developers., "Firebase Documentation: Cloud Firestore, Authentication, and Cloud

Messaging," <https://firebase.google.com/docs>, 2023., 2023.

[2] JetBrains, "Kotlin Programming Language Documentation.",

<https://kotlinlang.org/docs/home.html>, 2023.

[3] A. Developers., Material Design 3 Guidelines., <https://m3.material.io>, 2023.

[4] E. Union, "General Data Protection Regulation (GDPR)", <https://gdpr-info.eu>, 2018.

[5] A. Developers, "Room Persistence Library Guide,"

<https://developer.android.com/training/data-storage/room>, 2023.

[6] I. Square, "Retrofit: A Type-Safe HTTP Client for Android and Java.",

<https://square.github.io/retrofit>, 2023.